

Store Intelligence API

Offline Retail Analytics from CCTV Events

A complete challenge submission with a containerised FastAPI analytics service, event ingestion, real-time metrics, funnel analytics, anomaly detection, health checks, generated JSONL event log, tests, and a terminal dashboard.

Submission Snapshot

Repository:
github.com/Shrichackran/store-intelligence
Event log: outputs/events.jsonl
Runtime: docker compose up --build
Coverage: 81%
Dashboard: terminal metrics view

FastAPI

Docker

JSONL Events

SQLite

Tests 81%

Prepared by Shrichackran K M

Challenge Fit

MANDATORY DELIVERABLES MAPPED TO REPOSITORY

Required Output

Working, containerised Store Intelligence API with live metrics and event ingestion.

Critical Event Log

Generated JSONL event file committed at outputs/events.jsonl.

Documentation

README.md, DESIGN.md and CHOICES.md included in repository root.

Production Readiness

Docker Compose startup, idempotent ingest, structured API, tests and coverage.

Analytics API

/metrics, /funnel, /heatmap, /anomalies and /health return JSON.

AI Engineering

AI-assisted decisions documented with trade-offs and overrides.

- Submission format: Git repository link with DESIGN.md and CHOICES.md in the root.
- Event log deliverable: outputs/events.jsonl is included and follows JSONL format.
- The Docker API runtime was verified locally through docker compose up --build.

Problem and North Star Metric

TURNING OFFLINE STORES INTO MEASURABLE SESSIONS

Apex Retail has strong online analytics, but physical stores are a blind spot. The system converts raw CCTV-derived events and POS timing into actionable offline conversion analytics.

Who visited?

ENTRY / EXIT and visitor sessions feed traffic and unique visitor metrics.

Where did they engage?

ZONE_ENTER, ZONE_EXIT and ZONE_DWELL drive heatmap and dwell analytics.

North Star Metric

Offline conversion rate = visitors who completed purchase / total unique visitors in session window.

Where did they drop?

Funnel stages expose loss from entry to zone visit to billing to purchase.

Is operations healthy?

Queue and stale-feed warnings surface operational risks.

Are staff excluded?

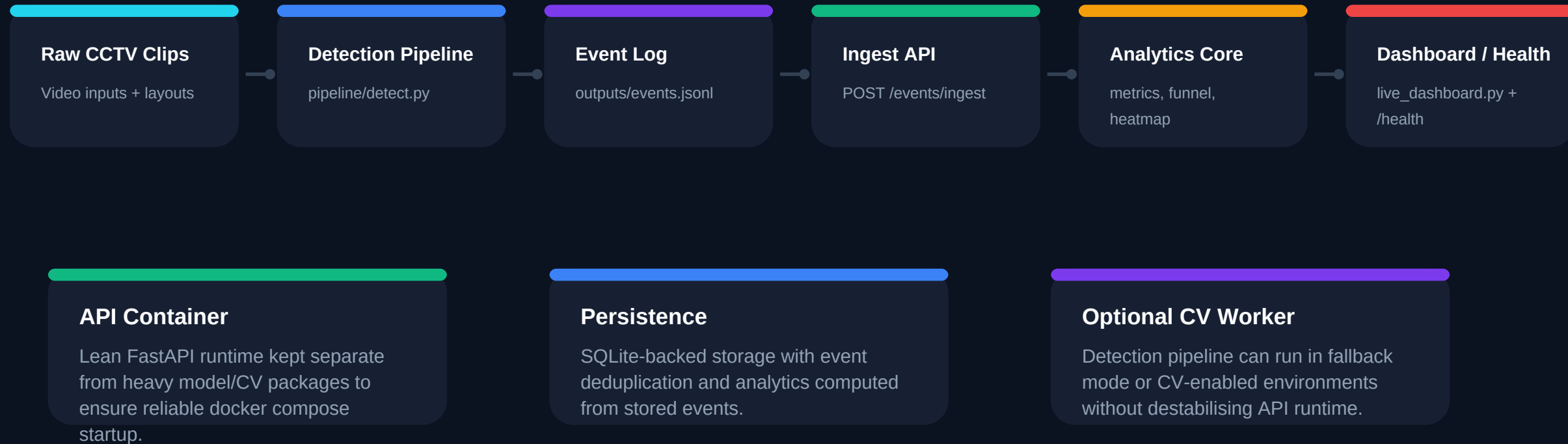
Metrics exclude is_staff=true to protect customer analytics quality.

What needs action?

Anomalies include suggested_action strings for store operators.

System Architecture

PIPELINE TO API TO LIVE ANALYTICS



Event Output and Schema Compliance

CRITICAL DELIVERABLE: VALID JSONL EVENT LOG

Committed Artifact

outputs/events.jsonl is included in the repository and should be used as the generated event log.

JSONL Format

Each line is one JSON object. The file can be validated line-by-line and ingested as a batch.

Sample-Compatible

Events include event_id, event_type, store_id, camera_id, event_timestamp, is_staff, track_id and zone fields.

Event log generation and verification

```
PYTHONPATH=. python pipeline/detect.py --mode fallback  
head -n 5 outputs/events.jsonl
```

```
git ls-files outputs/events.jsonl  
# expected: outputs/events.jsonl
```

Example JSONL line shape

```
{"event_id": "...",  
  "event_type": "entry",  
  "store_id": "ST1076",  
  "camera_id": "CAM 3 - ENTRY",  
  "event_timestamp": "2026-...",  
  "is_staff": false,  
  "track_id": 3162,  
  "zone_id": "ST1076_Z_CAM 3 - ENTRY"}
```

Intelligence API Surface

ENDPOINTS ALIGNED WITH SCORING REQUIREMENTS

POST /events/ingest

Batch ingest up to 500 events, validates, deduplicates by event_id and returns partial-success errors.

GET /stores/{id}/metrics

Unique visitors, conversion rate, dwell, queue depth, abandonment and purchase metrics.

GET /stores/{id}/funnel

Session-based funnel from entry to zone visit to billing queue to purchase.

GET /stores/{id}/heatmap

Zone visit and dwell data prepared for heatmap rendering.

GET /stores/{id}/anomalies

Queue spike, conversion drop and dead-zone style operational warnings.

GET /health

Service status, last event timestamp and STALE_FEED warning when lag exceeds threshold.

Validation Evidence

WHAT WAS VERIFIED BEFORE SUBMISSION

Docker

docker compose up --build completed and Uvicorn served on port 8000.

Tests

pytest with coverage passed: 13 tests and 81% coverage after configured scope.

Ingest

POST /events/ingest accepted events with 0 rejected in verified run.

Dashboard

dashboard/live_dashboard.py returned metrics, funnel, anomalies and health JSON.

Endpoints

/health, /docs, /metrics, /funnel, /heatmap and /anomalies returned JSON.

Event Log

outputs/events.jsonl generated, validated as JSONL and committed.

Reviewer verification commands

```
pytest --cov=app --cov=pipeline --cov-report=term-missing  
# 13 passed, coverage 81%  
curl http://localhost:8000/health  
python dashboard/live_dashboard.py
```

Production Readiness Choices

RELIABILITY FIRST, MODEL RUNTIME SECOND

Lean API Container

The API Docker image avoids PyTorch/Triton-sized dependencies, keeping docker compose reliable in constrained environments.

Separation of Concerns

Detection worker and API runtime are separated. The API remains stable while model dependencies can evolve independently.

Graceful Analytics

Zero traffic and zero purchase cases return safe numeric JSON instead of nulls or crashes.

Idempotent Ingest

Duplicate event_id values are skipped, making repeated calls safe for replay and retry workflows.

Health and Staleness

The health endpoint flags STALE_FEED when store event lag exceeds threshold.

AI Documentation

DESIGN.md and CHOICES.md explain AI-assisted decisions, trade-offs and overrides.

Reviewer Runbook

COMMANDS TO TEST THE SUBMISSION

Minimal run instructions

```
git clone https://github.com/Shrichackran/store-intelligence.git
cd store-intelligence
docker compose up --build

# second terminal
curl http://localhost:8000/health
curl http://localhost:8000/stores/ST1076/metrics
PYTHONPATH=. python pipeline/detect.py --mode fallback
head -n 5 outputs/events.jsonl
pytest --cov=app --cov=pipeline --cov-report=term-missing
python dashboard/live_dashboard.py
```

Submission Fields

Demo Link and Repository URL: <https://github.com/Shrichackran/store-intelligence> | Source Code: store-intelligence-submission.zip

Final Submission Package

WHAT REVIEWERS RECEIVE

- Repository URL: <https://github.com/Shrichackran/store-intelligence>
- Event log: outputs/events.jsonl
- README.md: setup, API usage, detection pipeline and output deliverables
- DESIGN.md: architecture overview and AI-Assisted Decisions
- CHOICES.md: model selection, schema design and API architecture decisions
- Dockerfile + docker-compose.yml: containerised API runtime
- Tests + .coveragerc: coverage above 70%
- dashboard/live_dashboard.py: terminal dashboard proof

Final Instruction

Submit the GitHub repository link in the platform. If the repo is private, invite purplletechchallenge2026@hackerearth.com before submitting.